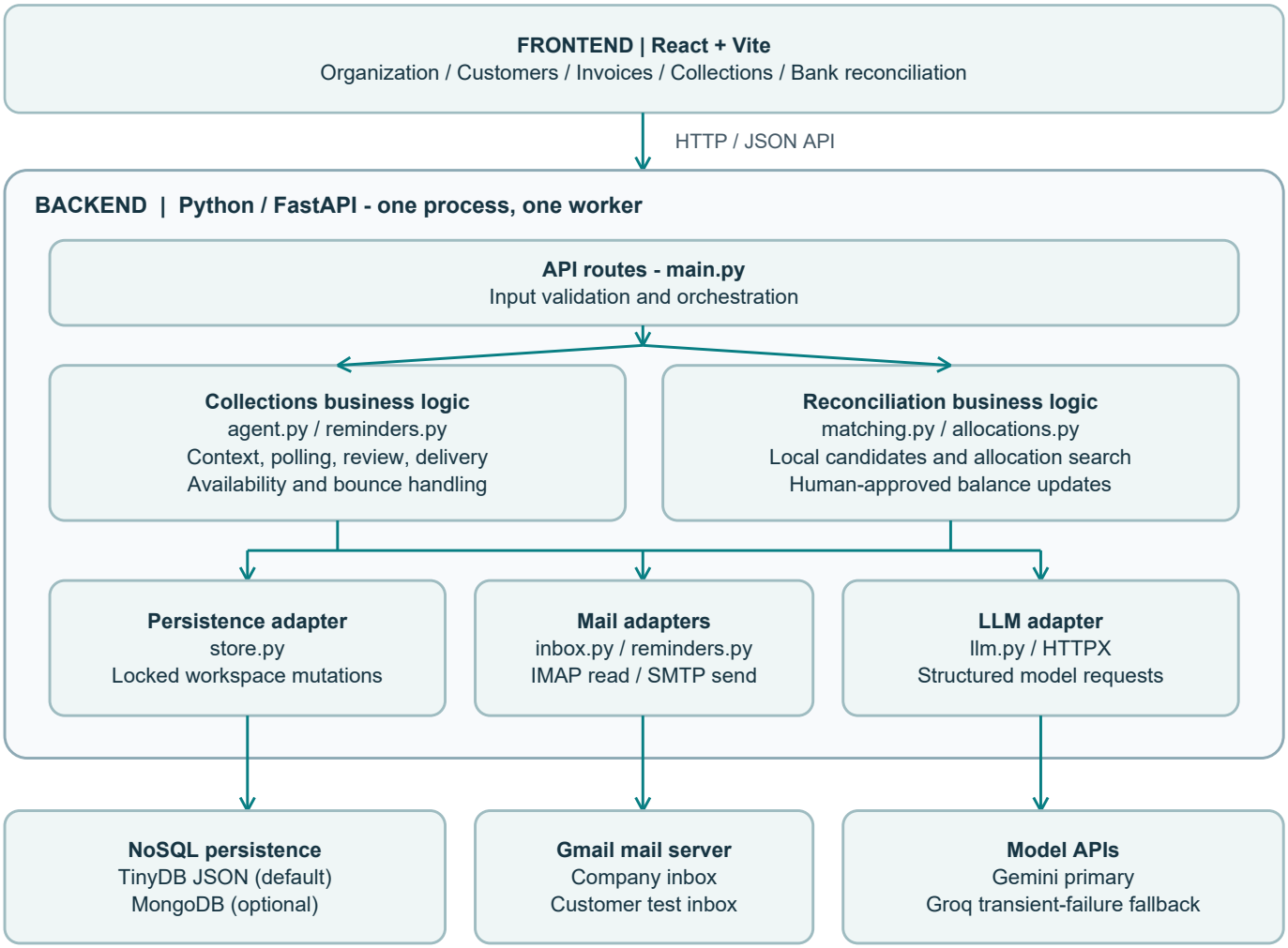
System architecture

A separate React client, a modular FastAPI backend and external services

Acme Collections helps AcmeCorp chase unpaid invoices and reconcile bank receipts. It loads nine fictional customers and 27 invoices. The UI monitors balances and conversations, sends reminders and collects human decisions. **A payment claim never closes an invoice; bank reconciliation approval does.**



Arrows represent backend calls; results return to the caller. Business modules share the adapters as needed. Database, mail and model credentials remain behind the API boundary. This is a modular application, not separate microservices.

Technology choices

Frontend: React 19, Vite 6, JavaScript and CSS; npm workspaces manage the client. **Backend:** Python, FastAPI and Uvicorn; HTTPX calls providers. **Storage:** TinyDB for a zero-server local setup, or PyMongo + optional Docker MongoDB. **Transport:** Gmail SMTP with STARTTLS and IMAP over TLS. Environment variables configure credentials and provider models.

How the layers work together

A reminder preview reads current invoice balances through the backend. Incoming mail triggers customer-specific LLM processing and stores the result. Separately, a CSV receipt produces local match suggestions. Approval records cash/TDS, reduces balances and marks zero balances paid. The next reminder and agent context read the updated state, so settled invoices are no longer chased. README documents setup and API routes.

Database and implemented features

Persistent workspace design, customer-scoped processing and review controls

Document design and relationships

The database contains **one workspace document**, not one collection per entity. TinyDB stores it in **backend/storage/workspace.json**; MongoDB stores it in the **workspace** collection with **_id = demo**. The supplied JSON files seed a new database. The following are nested entities and logical references, not SQL foreign keys:

organization: company contacts, bank details and billing policies.
customers[]: customer_id, name/aliases, contact email and reminder-pause state.
invoices[]: invoice_number, customer_id, dates, currency, subtotal, amount_paid, tds_adjustment, balance_due and draft/sent/paid status.
transactions[] + payments[]: imported bank receipts and review decisions; approved payment allocations link transaction_id to invoice_number and store cash/TDS.
conversations[]: customer-linked threads with nested incoming/outgoing messages, message IDs, references, model decisions and delivery/review information.
mailbox + activity[]: account, UIDVALIDITY, last UID, deduplication/check metadata and recorded application actions.

Logical links: **customer -> many invoices and conversations**; **transaction -> payment allocations -> invoices**. Reads return copies. A re-entrant lock protects validate-then-write mutations. TinyDB replaces its JSON file atomically; MongoDB replaces one document. This simplifies consistency for the small dataset but requires one backend worker and is not a scalable multi-user database design.

Implemented user workflows

Workspace and reminders: organization, editable customer emails, invoices, outstanding balances and derived overdue indicators. Select one or multiple sent/unpaid invoices, preview one consolidated reminder, then send it. Sending revalidates current balances, recipient restrictions and pauses; identical accepted/uncertain sends are guarded against duplication.

Inbox and AI agent: manual Check inbox now plus automatic polling 60 seconds after each completed cycle. UID tracking avoids replay; sender and threading evidence identify the customer. Context contains organization policies/payment details, this customer's eligible invoices, recent thread, incoming text and date context. Gemini classifies/drafts; selected transient failures can use Groq. Routine automatic replies require confidence at least 0.90, allowed intent and backend validation; financial facts are rendered from current records.

Human review and exceptions: extensions, disputes, legal issues and unsafe/failed generations require attention; a draft may be empty and the reviewer can write/edit it. Normal mail and attention items are separated. Payment claims are acknowledged without changing balances. Out-of-office notices pause reminders after local date checks, without changing invoice due dates. Bounce reports receive no AI reply; confirmed failures block reminders to the failed address.

Reconciliation and persistence: CSV upload/sample import, duplicate detection, scored suggestions, combined allocations, manual selection, approval/rejection and TDS-versus-open-balance review. Approval is revalidated and applied as one workspace mutation. Dashboard refresh is every 10 seconds. Reset restores seed business data and clears conversations/payments while retaining mailbox tracking; real Gmail messages and credentials are not deleted.

Validation, security and remaining scope

Verified: 66 backend regression tests and a successful frontend build. Secrets stay in backend .env; customer-scoped context, structured output checks and restricted sending reduce risk but do not prove prompt-injection immunity. Human-reviewed free text is not fully fact-validated. The prototype has no production login, uses Gmail and one configured test payer, and has no recurring reminder scheduler. **Two bonus items are implemented: bonus 2 (one payment covering several invoices) and bonus 3 (out-of-office replies and bounced emails).** Bonus 1 (payment-email-to-bank-transaction linking) remains unimplemented. Unknown unrelated senders are skipped; known-thread unknown senders require review.

Matching, scores and assumptions

Deterministic local evidence - suggestions never authorize payment

1 / Eligibility, references and company identification

Match credits only to sent invoices with balance > 0, equal currency and invoice_date <= receipt date. Due date is not a filter. Extract bounded INV references from narration/reference fields; unknown, ineligible or cross-customer references require review. Normalize names to lowercase ASCII, remove transfer/legal terms and expand aliases (e.g. hosp -> hospitality). Compare all customer names/aliases, including companies without eligible invoices.

T = maximum SequenceMatcher ratio between sorted token sets and their shared-token comparisons. **C** = cosine similarity of character 3/4-gram TF-IDF vectors. Gram weight = **(1 + ln f) x [1 + ln((1 + N)/(1 + df))]**, with count f, document frequency df and N name/alias entries. **P = 0.55T + 0.45C**; exact normalized name phrases score 1. Keep the best alias per company. Without a reference require **P >= 0.72** and a **0.12** runner-up lead. A reference can work with weak name evidence; strong conflicting/ambiguous payer evidence blocks it.

2 / Exact single-invoice scoring

S = min(99, base + amount evidence)
Base: 70 for an invoice reference; otherwise round(55P).
Amount evidence: +25 exact current balance; +20 possible INR TDS (balance - cash = 10% of subtotal); +5 partial payment. Overpayments are excluded.
Use integer cents. If the top two invoice scores differ by < 10, return ambiguity and retain candidates, rather than choose an invoice. Scores rank evidence; they are not calibrated probabilities.

Example: referenced balance INR 354,000, subtotal INR 300,000, receipt INR 324,000: score **70 + 20 = 90**. A reviewer chooses INR 30,000 TDS closure or leaves the difference open. Exact referenced cash scores **95**; exact cash with P = 1 and no reference scores **80**. Equal-amount invoices can remain ambiguous.

3 / Bonus 2: one payment covering several invoices

The matcher suggests invoice combinations and splits the receipt on approval. For a clear payer without references, bounded depth-first subset search explores **skip / full balance / INR TDS closure** per invoice. With multiple references, include all referenced invoices and enumerate full/TDS closures plus at most one residual partial invoice (zero TDS on that partial). Limits: **18 invoices, 100,000 nodes, 20 results**; incomplete searches are flagged. It searches combinations, not every ordering.

Combined score: unique, non-truncated unreferenced multi-invoice settlement = **80**; unique, non-truncated all-referenced closure with no partial = **95**. Ambiguous/truncated combined results or referenced partial alternatives = **0**. These fixed combined scores do not use the single-invoice formula. Approval checks current balances, eligibility, one customer, cash sum = receipt, valid TDS and prior approval before recording allocations and reducing balances.

Example (illustrative): referenced balances A = 1,000, B = 2,000, C = 3,000 and cash = 4,000 can produce A fully paid (1,000), B fully paid (2,000), C partially paid (1,000; 2,000 remains). Other valid splits are shown for human selection. For INR TDS closures, cash plus the separately recorded 10%-of-subtotal deduction must equal the balance. Approval records one payment allocation per invoice atomically; only zero balances become paid.

4 / Why this combination, and assumptions

References and filters provide precise evidence; token comparisons tolerate word order/noise, while character TF-IDF handles spelling fragments. Amount-only/FIFO can pick the wrong equal-value invoice. Levenshtein/Jaro-Winkler alone are sensitive to noisy whole strings; Jaccard misses typos. Embeddings could help unfamiliar aliases but need evaluation and another model. Local subset search is reproducible and avoids LLM cost/latency: **no bank statement is sent to an LLM**. Thresholds are heuristic, not proven optimal.

Payer assumption: absent invoice references, narration must identify the company or a known alias.
Personal-account/third-party payments are not reliably resolved by amount and need manual review or a future verified mapping; this is not bank-account ownership verification. A valid reference can still identify a candidate. **Financial assumptions:** one customer/currency per receipt, no FX/fee inference or automatic overpayment allocation; TDS is the assignment's INR-only 10%-of-subtotal rule. Unreferenced combinations cover full/TDS settlements; referenced suggestions support at most one partial. Other splits require intervention.

Basis: Sonic AI assignment sections 4-8 and 10; backend/app matching, allocations, store, agent and mail modules. Setup and API details: README.md.